Microsoft Agent Framework

Extensive training material for architects, developers, and delivery teams

Edition: April 2026
Scope: Framework fundamentals, tools, sessions, memory, workflows, hosting, governance, and hands-on labs

Use this pack as a self-study guide, instructor manual, and workshop handout.

Topic	Details
Audience	Solution architects, AI engineers, full-stack developers, platform teams, and technical leads evaluating or adopting Microsoft Agent Framework.
Recommended background	Python or C#, REST/SDK basics, LLM concepts, cloud authentication, and general familiarity with Azure OpenAI or Microsoft Foundry.
Learning outcome	By the end of the course, learners should be able to design, build, debug, and productionize an agent or multi-agent workflow using Agent Framework and related Microsoft services.

How to use this training pack

- As a 1-day workshop: cover Modules 1-8, run Labs 1-3, and use the governance checklist during solution review.
- As a 2-day bootcamp: include all labs, add design review exercises, and compare deployment choices such as local hosting versus Foundry Agent Service.
- As self-study: read each module sequentially and complete the checkpoint questions before moving to the next section.

Suggested delivery plan

Module	Duration	Mode	Outcome
1. Foundations	45 min	Lecture	Understand what Agent Framework is and where it fits.
2. Core runtime	60 min	Lecture + demo	Build a first agent, add tools, and stream responses.
3. State and memory	45 min	Lecture + lab	Use sessions and context providers correctly.
4. Workflows	60 min	Lecture + lab	Coordinate steps and agents with type-safe

			orchestration.
5. Integration and hosting	45 min	Lecture	Connect providers, MCP tools, and managed hosting.
6. Observability and quality	45 min	Lecture	Trace, evaluate, and monitor production behavior.
7. Security and governance	30 min	Lecture	Apply enterprise controls and safe tool design.
8. Solution patterns	30 min	Workshop	Translate knowledge into real project architecture.

Module 1 - Foundations and mental model

Key idea: Agent Framework is the open-source application framework; Foundry Agent Service is the managed runtime platform.

Microsoft Agent Framework is an open-source SDK and runtime for building, orchestrating, and deploying AI agents and multi-agent workflows in Python and .NET. Microsoft positions it as a unified foundation that brings together production-ready ideas from Semantic Kernel and AutoGen while adding a consistent model, tool, workflow, and hosting story.

At a conceptual level, the framework gives you two connected layers:

- Application layer: agents, tools, sessions, context providers, workflows, middleware, and integration adapters.
- Platform layer: model providers, identity, deployment, observability, governance, and optional managed hosting through Foundry Agent Service.

What problems it solves

- Moves from one-off prompt demos to structured agent applications with reusable abstractions.
- Lets teams add tools, multi-turn memory, workflow control, and human-in-the-loop behavior without reinventing orchestration code.
- Provides a path from local experimentation to production deployment and monitoring.
- Supports multiple providers, which reduces lock-in and helps teams evolve their inference strategy.

Essential vocabulary

Topic	Details
Agent	A unit of AI behavior that combines instructions, model access, tools, and execution logic to fulfill a task.
Session	Conversation or run state used to preserve context across multiple turns.
Context provider	A component that injects useful context before inference and can store information after the run.

Middleware	Interception points for cross-cutting logic such as logging, validation, approvals, policy checks, or transformation.
Workflow	A graph or sequence that connects agents and functions into a controlled business process.
MCP	Model Context Protocol, an open standard for exposing tools and contextual data to model-powered applications.
Foundry Agent Service	Managed platform for building, deploying, scaling, tracing, evaluating, and monitoring agents.

Architecture principle

Use an agent when the system benefits from model reasoning and tool selection. Use a workflow when you need predictability, branching, checkpoints, or explicit control over multi-step execution. In practice, successful production solutions often combine both: an agent performs reasoning inside a workflow that enforces business boundaries.

Module 2 - Core runtime building blocks

The Agent Framework overview highlights several foundational building blocks: model clients, an agent session for state management, context providers for memory, middleware for interception, and MCP clients for tool integration. These are the components most teams touch first when building a real solution.

Agent types and providers

Agent Framework supports several agent types and provider integrations. Providers determine how your agent reaches a model or external runtime. Current documentation highlights support for Microsoft Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama, GitHub Copilot integrations, Copilot Studio connections, and custom providers.

Typical build sequence

1. Choose the provider and authentication model.
2. Define the agent instructions and expected behavior.
3. Attach tools the model may call.
4. Enable sessions for multi-turn conversation.
5. Add context providers or memory if the agent must personalize or persist knowledge.
6. Insert middleware for policy, diagnostics, or approvals.
7. Wrap the agent in a workflow if the task spans multiple steps or actors.

First-agent reference flow

The official quick-start walks through a progression: create your first agent, add tools, maintain multi-turn conversations with sessions, inject memory and persistence with context providers, compose workflows, and finally host the agent. That progression is a good teaching sequence because it mirrors the maturity path of most real implementations.

Python example - minimal agent skeleton

```
from agent_framework import Agent

# Pseudocode for teaching purposes - align package names with the latest docs/sample repo.
agent = Agent(
  name="TravelAssistant",
  instructions="Help users compare options and ask clarifying questions when needed.",
)

result = await agent.run("Plan a 2-day trip to Bengaluru focused on food and architecture")
print(result.output_text)
```

Training note: package names and constructor signatures should always be validated against the latest Microsoft Learn sample or GitHub repo before coding in production, because preview releases can change quickly.

Tool design guidance

- Expose narrow, meaningful tools rather than one huge do-everything function.
- Use descriptive names and concise docstrings so the model can select tools reliably.
- Make dangerous tools idempotent or approval-gated where possible.
- Return structured data that downstream logic can inspect instead of fragile prose-only results.

Module 3 - Sessions, context providers, and memory

A common mistake is to treat memory as a single feature. In Agent Framework, there is an important distinction between short-term conversational state and durable contextual memory.

Sessions

Sessions maintain conversation state across turns. In training terms, this is your short-term working memory. It preserves prior messages and allows the model to continue a conversation without resending all user context manually each time.

Context providers

Context providers add or extract context during the agent pipeline. Microsoft Learn notes that a context provider can inject messages, tools, or instructions before invocation, and can store information afterward. It can even extend middleware for the current invocation. One critical implementation rule from the docs is that the provider instance should not keep session-specific state on itself; session-specific data should live in the proper state mechanism, not in the provider object.

Memory strategy model

Topic	Details
Turn memory	Messages inside the current run or short conversation window.
Session memory	Conversation state carried across turns for the same user or thread.
Durable contextual memory	Facts or summaries persisted and re-injected later through context providers or memory services.
External enterprise knowledge	Data retrieved from systems of record, vector stores,

	APIs, or search indexes rather than memorized inside the model context.

Good practice

- Persist only what creates future value, such as preferences, workflow state, or approved summaries.
- Do not treat memory as a substitute for authoritative systems of record.
- Summarize long histories before re-injection to control token growth.
- Store provenance or source metadata when durable context will influence decisions.

Lab 1 - Personalization with a context provider

- Goal: teach the agent to remember user preferences across multiple turns.
- Task: create a context provider that stores a user preference such as preferred city, language, budget, or document style in session state.
- Stretch task: add a guard so the provider stores only explicit user preferences, not accidental or sensitive data.
- Success criteria: the agent adapts its answer in turn 2 or 3 without the user re-supplying the preference.

Module 4 - Middleware and pipeline architecture

The pipeline architecture is where Agent Framework becomes especially enterprise-friendly. Microsoft documents three middleware layers: agent run middleware, function calling middleware, and chat client middleware. This separation helps teams place controls where they belong instead of building one tangled interception layer.

Topic	Details
Agent run middleware	Inspect or modify overall run input/output. Good for trace enrichment, approvals, request shaping, or result post-processing.
Function calling middleware	Inspect or modify tool inputs/outputs. Good for authorization, rate limiting, validation, masking, and retry logic.
IChatClient middleware	Intercept low-level inference calls. Good for model request diagnostics, policy wrappers, token accounting, or provider-specific behavior.

Common enterprise uses

- Redact sensitive prompts before they reach the model.
- Enforce tool-level authorization checks.
- Inject correlation IDs and trace attributes into every run.
- Add fallback behavior or uniform error translation for downstream systems.
- Transform raw tool results into a controlled schema before they re-enter model context.

Design warning

Avoid hiding core business logic inside middleware. Middleware is strongest for policy and infrastructure concerns. If you put the main workflow semantics there, debugging becomes hard and the design loses clarity.

Module 5 - Workflows and multi-agent orchestration

Microsoft Agent Framework Workflows focuses on type-safe orchestration for intelligent automation. The docs describe graph-based workflows that can blend AI agents with business logic while supporting checkpointing and human-in-the-loop scenarios. This is the right abstraction when you need explicit control rather than open-ended autonomous behavior.

When to use workflows

- A task has known stages such as collect -> analyze -> review -> approve -> publish.
- Different agents have specialized roles such as planner, writer, reviewer, and compliance checker.
- The process needs branching, retries, escalation, or checkpoints.
- Human approval is required before a tool can take an external action.

Sequential writer-reviewer example

The current Learn samples show an agents-in-workflows example in which a Writer agent drafts content and a Reviewer agent provides feedback, connected in a sequential workflow with streaming updates. This is a strong teaching pattern because it shows specialization without overwhelming learners with too many branches.

Workflow design heuristics

- Use a workflow node for deterministic transitions and side effects.
- Use an agent node when reasoning or language generation is the value-adding step.
- Keep node contracts typed and explicit.
- Checkpoint after expensive or approval-sensitive stages.
- Prefer smaller specialized agents over one giant generalist when accountability matters.

Lab 2 - Build a writer-reviewer workflow

- Use one shared model client and instantiate two specialized agents: Writer and Reviewer.
- Pass a user brief into the workflow and stream the output from both stages.
- Add a final deterministic node that formats the approved answer into a JSON or markdown structure.
- Stretch task: add a human approval gate before publication.

Module 6 - MCP and external tool integration

Model Context Protocol is becoming an important tool integration pattern. Microsoft Learn states that Foundry agents can connect to MCP server endpoints, extending capabilities with external tools and data sources, while Agent Framework also supports MCP integration for local or remote tool usage.

Why MCP matters

- Creates a standard way to expose tools and context to agentic applications.
- Reduces the need for one-off custom adapters for every service.
- Helps organizations centralize tool governance and reusability.
- Creates a bridge between local development and managed agent platforms.

Practical governance guidance

- Review third-party MCP servers before allowing enterprise data to flow through them.

- Grant only the subset of tools needed for the use case.
- Prefer read-only tools during early pilot phases.
- Track tool usage in traces so that risky behaviors can be audited.

Lab 3 - Add safe external tools

- Choose one low-risk retrieval tool and one action tool.
- Require confirmation or approval before the action tool executes.
- Capture and compare the trace data for a successful tool invocation versus a blocked one.
- Document what the agent should do when the tool is unavailable or returns partial results.

Module 7 - Hosting and deployment choices

Your deployment model should match your control needs. Foundry Agent Service is the managed option described by Microsoft as a fully managed platform for building, deploying, and scaling AI agents. It supports prompt agents, workflow agents, and hosted agents. Hosted agents let you deploy code-based agents built with Agent Framework, LangGraph, or your own containerized implementation.

Topic	Details
Local or self-hosted runtime	Best for prototyping, deep debugging, or environments where you need direct control over runtime shape and infrastructure.
Foundry prompt agents	Best for simple agents with low code and fast prototyping.
Foundry workflow agents	Best for declarative orchestration, branching, approvals, and repeatable automation.
Foundry hosted agents	Best for custom code, complex orchestration, multi-agent logic, and bespoke tool integration.

Foundry lifecycle

Microsoft describes a full lifecycle of create, test, trace, evaluate, publish, and monitor. For training, this is useful because it reinforces that deployment is not the end state; observability and evaluation are first-class operating requirements.

Deployment checklist

- Pin package versions and document breaking changes across preview upgrades.
- Separate dev, test, and prod configurations for tools and secrets.
- Use managed identity or equivalent secure identity mechanisms instead of embedded credentials.
- Define clear rollback criteria when updating prompts, tools, or workflow logic.
- Load test tool-heavy scenarios, not just pure chat scenarios.

Module 8 - Observability, quality, and evaluation

Production AI systems need more than logs. Microsoft Foundry describes three core observability capabilities: evaluation, monitoring, and tracing. Together, these help teams measure quality, inspect execution flow, and monitor reliability in production.

Topic	Details
Evaluation	Measure output quality, safety, reliability, groundedness, relevance, task completion, and tool-call accuracy.
Monitoring	Track latency, tokens, errors, quality thresholds, and operational behavior via dashboards and alerts.
Tracing	Understand execution flow across model calls, tool usage, agent decisions, and dependencies using OpenTelemetry-aligned tracing.

Metrics that matter

- Task success rate or business completion rate.
- Tool call correctness and unnecessary tool usage rate.
- Latency per workflow stage, not only end-to-end latency.
- Token and cost profile by scenario.
- Safety incidents, harmful output rate, and policy-block rate.
- Human override frequency for approval-gated tasks.

Evaluation strategy

Start with a baseline dataset that represents real tasks, then score changes to prompts, tools, and workflow policies before promotion. For RAG-style agents, add groundedness and relevance checks. For action agents, add task-completion and tool-accuracy checks. Treat regressions as release blockers, not post-production surprises.

Module 9 - Security, governance, and responsible use

Enterprise-ready agent design requires governance at model, prompt, tool, and infrastructure levels. Foundry documentation emphasizes enterprise capabilities such as identity, observability, and private networking for supported agent types. Agent Framework documentation also surfaces middleware and integration points that are useful for policy enforcement.

Security checklist

- Classify each tool by risk: read-only, internal action, external action, or privileged action.
- Require approval for high-impact actions such as ticket closure, financial update, or production change.
- Apply least privilege to every identity used by the agent or tool.
- Scrub sensitive data from traces or prompts when possible.
- Document data residency and third-party MCP implications before rollout.
- Define a kill switch or policy block mechanism for unsafe tool behavior.

Responsible design pattern

A strong pattern is: deterministic workflow shell -> specialized agent reasoning -> validated tools -> approval gate -> trace and evaluation -> controlled publication. This keeps autonomy where it helps and control where the business requires certainty.

Module 10 - Reference architecture patterns

Topic	Details
Internal knowledge assistant	Agent Framework + search/retrieval + tool calls for citation lookup + workflow for escalate-to-human.
Document generation assistant	Planner agent + writer agent + reviewer agent + policy middleware + publish workflow.
Operations copilot	Read-only tools for telemetry and ticketing, approval-gated action tools, and strong tracing/monitoring.
Case management orchestrator	Workflow-driven process with checkpoints, handoffs, and specialized agents for triage, enrichment, and response drafting.

Pattern selection guide

- Use a single agent for constrained, low-risk conversational tasks.
- Use multiple specialized agents when role clarity improves quality or accountability.
- Use a workflow when business policy or integration sequencing matters more than open-ended autonomy.
- Use managed hosting when platform operations and enterprise controls outweigh raw infrastructure flexibility.

Workshop exercises and discussion prompts

- Which parts of your target use case require agentic reasoning, and which should remain deterministic?
- What is the minimum tool set that still delivers user value?
- Which facts should be remembered, retrieved, or ignored?
- Where do you need approval gates?
- How will you know the agent is getting better over time?
- What traces or metrics will you inspect first when behavior goes wrong?

Implementation starter checklist

- Select language track: Python or .NET.
- Choose provider and authentication method.
- Define the agent objective, success criteria, and tool boundaries.
- Create a first working single-agent flow.
- Add sessions and explicit state handling.
- Introduce one context provider for durable context.
- Add middleware for tracing and policy.
- Convert the solution into a workflow if the process is multi-step.

- Instrument evaluation and monitoring before production rollout.
- Document governance, versioning, and rollback approach.

Frequently asked questions

Topic	Details
Is Agent Framework the same as Foundry Agent Service?	No. Agent Framework is the application framework/SDK. Foundry Agent Service is the managed platform that can host and operate agents, including hosted agents built with Agent Framework.
When should I use a workflow instead of a single agent?	Use workflows when you need explicit sequencing, branching, approvals, checkpoints, or clear stage contracts.
Is memory always a vector store?	No. In many cases memory is session state, structured preferences, summaries, or context-provider logic rather than semantic search.
Why use middleware?	To add cross-cutting concerns such as security, logging, validation, retries, or transformation without changing the core agent logic.
What should I monitor first?	Start with task success, tool correctness, latency, token/cost, safety incidents, and trace visibility across key workflow nodes.

Official reference sources used for this training material

- Microsoft Learn - Agent Framework documentation hub: https://learn.microsoft.com/en-us/agent-framework/
- Microsoft Learn - Agent Framework overview: https://learn.microsoft.com/en-us/agent-framework/overview/
- Microsoft Learn - Get started with Agent Framework: https://learn.microsoft.com/en-us/agent-framework/get-started/
- Microsoft Learn - Workflows: https://learn.microsoft.com/en-us/agent-framework/workflows/
- Microsoft Learn - Context providers: https://learn.microsoft.com/en-us/agent-framework/agents/conversations/context-providers
- Microsoft Learn - Middleware: https://learn.microsoft.com/en-us/agent-framework/agents/middleware/
- Microsoft Learn - MCP tools with agents: https://learn.microsoft.com/en-us/agent-framework/agents/tools/local-mcp-tools
- Microsoft Learn - Foundry Agent Service overview: https://learn.microsoft.com/en-us/azure/foundry/agents/overview
- Microsoft Learn - MCP tool in Foundry Agent Service: https://learn.microsoft.com/en-us/azure/foundry/agents/how-to/tools/model-context-protocol
- Microsoft Learn - Observability in generative AI: https://learn.microsoft.com/en-us/azure/foundry/concepts/observability
- GitHub - microsoft/agent-framework: https://github.com/microsoft/agent-framework

- Microsoft Foundry blog - Introducing Microsoft Agent Framework: https://devblogs.microsoft.com/foundry/introducing-microsoft-agent-framework-the-open-source-engine-for-agentic-ai-apps/

Appendix - Instructor notes

- Emphasize the difference between reasoning and orchestration. Learners often overuse agents where workflows are more appropriate.
- Show at least one trace screenshot or live trace walkthrough during delivery, because it anchors debugging discussions in reality.
- During labs, ask participants to write down failure modes before they code. This improves design quality.
- If time is short, prioritize Labs 1 and 2; they cover the most transferable concepts.